

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет прикладной математики, информатики и механики

Кафедра программного обеспечения
и администрирования информационных систем

Направление 02.03.03 Математическое обеспечение
и администрирование информационных систем

Отчет

по учебной практике (проектной)

Тема

Бинарные деревья. Бинарные деревья поиска (цепочное и сплошное
представление

Обучающийся

3 курс, 91 группа, Миа Музамил Ахмад

Руководитель от кафедры

Артемов М. А.

Воронеж 2026

Содержание

Содержание.....	2
Введение.....	4
История развития	6
Целевое назначение и области применения	8
1. Теоретическая часть.....	10
1.1 Основные понятия о деревьях	10
1.2 Основные понятия о бинарных деревьях	14
1.2.1 Определение	14
1.2.2 Основные свойства	14
1.2.3 Применение	15
1.2.4 Основные операции	15
1.3 Бинарные деревья поиска	16
1.3.1 Определение	16
1.3.2 Инвариант BST.....	17
1.3.3 Основные операции	18
1.3.3.1 Вставка (<i>Insertion</i>).....	18
1.3.3.2 Поиск	19
1.3.3.3 Удаление (<i>Deletion</i>).....	20
1.3.4 Сложность операций	21
1.3.5 Представление бинарных деревьев поиска (BST).....	22
1.3.5.1 Цепочное представление	22
1.3.5.2 Сплошное представление.....	24
1.3.6 Сравнительный анализ	26
1.4 Методы организации и хранения бинарных деревьев поиска	28

1.4.1 Общие принципы организации BST	28
1.4.2 Критерии выбора способа хранения	28
1.4.3 Эволюция методов хранения	29
1.4.4 Сравнение по ключевым параметрам	29
1.4.5 Оптимизация хранения	31
1.4.6 Современные тенденции	31
1.4.7 Практические рекомендации по организации	32
ПРИЛОЖЕНИЕ 1	33
Промпт для генерации теоретического материала по бинарным деревьям поиска	33
ПРИЛОЖЕНИЕ 2	35
Промпт для генерации программного кода цепочного представления BST	35
ПРИЛОЖЕНИЕ 3	43
Промпт для генерации программного кода сплошного представления BST	43
ПРИЛОЖЕНИЕ 4	48
Промпт для генерации сравнительного анализа и вывода рекомендаций	48
ПРИЛОЖЕНИЕ 5	50
Промпт для генерации истории развития бинарных деревьев.....	50
ПРИЛОЖЕНИЕ 6	52
Промпт для генерации списка использованных источников	52

Введение

Бинарные деревья являются одной из фундаментальных структур данных в информатике, широко применяемых для организации и хранения иерархических данных. Они используются в различных областях: от файловых систем и баз данных до компиляторов и алгоритмов искусственного интеллекта. Особое место среди них занимают бинарные деревья поиска (BST), которые обеспечивают эффективное выполнение операций поиска, вставки и удаления элементов за логарифмическое время $O(\log n)$ в сбалансированном состоянии, что делает их незаменимыми при работе с динамическими наборами упорядоченных данных.

В зависимости от способа физического хранения в памяти компьютера различают два основных представления бинарных деревьев: **цепочное** (динамическое, на основе указателей) и **сплошное** (на основе массива). Цепочное представление предполагает создание отдельных узлов в динамической памяти с ссылками на потомков, что обеспечивает гибкость при изменении структуры дерева, но требует дополнительных затрат памяти на хранение указателей и может приводить к фрагментации памяти. Сплошное представление использует непрерывный массив, где положение узла вычисляется по индексу с использованием формул $2*i+1$ и $2*i+2$ для левого и правого потомков соответственно. Такой подход дает преимущества в скорости доступа и локальности данных, однако может приводить к неэффективному использованию памяти при несбалансированных деревьях.

Выбор между этими подходами существенно влияет на производительность программы, потребление памяти и сложность реализации. Понимание особенностей каждого из них необходимо для принятия обоснованных решений при проектировании программных систем. Кроме того, в современной разработке важным навыком становится умение эффективно взаимодействовать с AI-ассистентами, формулируя корректные промпты для генерации программного кода. Цель данной работы — изучение

теоретических основ бинарных деревьев поиска, практическая реализация обоих способов представления, их сравнительный анализ, а также освоение навыков промпт-инжиниринга для автоматизации процесса разработки.

История развития

Бинарные деревья как структура данных начали использоваться в вычислительной технике в начале 1950-х годов, когда исследователи осознали, что бинарный поиск может быть эффективно представлен в виде бинарного дерева . В этот период активно развивались электронные вычислительные машины, что создало потребность в структурах данных для эффективной организации и поиска информации в памяти с ограниченными ресурсами .

Бинарное дерево поиска (BST) было разработано в 1960 году. Его создание приписывается Конвею Бернерсу-Ли и Дэвиду Уилеру, которые использовали эту структуру для хранения маркированных данных на магнитных лентах . Независимо от них, аналогичный алгоритм был открыт несколькими исследователями, включая П.Ф. Уиндли, Эндрю Дональда Бута, Эндрю Колина и Томаса Н. Хиббарда . Один из самых ранних и популярных алгоритмов BST был предложен Хиббардом .

Основная проблема ранних бинарных деревьев поиска заключалась в том, что при произвольном порядке вставки узлов дерево могло вырождаться и терять сбалансированность, что приводило к ухудшению времени поиска до $O(n)$. Для решения этой проблемы началась разработка **самобалансирующихся** вариантов BST .

Первым самобалансирующимся бинарным деревом поиска стало **AVL-дерево**, изобретённое в 1962 году советскими математиками Георгием Адельсоном-Вельским и Евгением Ландисом . Оно обеспечивало логарифмическое время выполнения операций поиска, вставки и удаления за счёт строгого контроля разницы высот поддеревьев .

В 1970 году Джон Хопкрофт предложил **2-3-дерево** — структуру, в которой каждый узел мог иметь два или три потомка, что обеспечивало балансировку без сложных операций вращения .

В 1971 году Байер и МакКрейт разработали **В-деревья**, обобщившие идею 2-3-деревьев на узлы с большим количеством потомков, что сделало их особенно эффективными для работы с дисковыми хранилищами и базами данных .

В 1972 году Рудольф Байер представил **симметричные бинарные В-деревья** (позднее получившие название ****красно-чёрных деревьев****) . Современный вариант красно-чёрного дерева был популяризирован в 1978 году Леонидасом Гуибасом и Робертом Седжвиком . Эти деревья использовали дополнительный бит цвета (красный/чёрный) для поддержания приблизительной сбалансированности и стали стандартной реализацией для многих библиотек и систем, включая Java TreeMap и C++ .

В 1970-х годах также появились ****весовые сбалансированные деревья**** (Nievergelt и Reingold), хранящие размеры поддеревьев в узлах, что позволяло эффективно реализовывать операции порядковой статистики .

Дальнейшее развитие структуры продолжилось в 1994 году, когда Питер Фенвик описал **дерево Фенвика** (Binary Indexed Tree) — структуру для эффективного выполнения операций над массивами с возможностью изменения элементов и вычисления ассоциативных операций на отрезках . В 2008 году Седжвик создал левосторонние красно-чёрные деревья, упростившие реализацию балансировки .

Целевое назначение и области применения

Бинарные деревья поиска (BST) представляют собой универсальную структуру данных, предназначенную для эффективного хранения, поиска и обработки упорядоченных динамических наборов данных. Их основное целевое назначение — обеспечение быстрого доступа к элементам, вставки и удаления с логарифмической сложностью $O(\log n)$, что делает их оптимальным выбором для систем, работающих с большими объемами динамически изменяющихся данных.

Основные области применения:

1. Базы данных и системы управления данными

- Индексирование записей для ускорения поиска.
- Реализация первичных и вторичных индексов.
- Обеспечение уникальности ключей.

2. Файловые системы

- Организация каталогов и файлов (иерархическая структура).
- Управление метаданными файлов.
- Реализация древовидных файловых систем (ext4, NTFS).

3. Компиляторы и интерпретаторы

- Построение синтаксических деревьев разбора.
- Представление абстрактного синтаксического дерева (AST).
- Оптимизация кода и анализ выражений.

4. Операционные системы

- Управление памятью (диспетчеры памяти).
- Планирование процессов.
- Организация системных вызовов.

5. Криптография и сетевая безопасность

- Хранение и проверка цифровых сертификатов.
- Управление ключами в PKI-системах.
- Реализация деревьев Меркла в блокчейн-технологиях.

6. Алгоритмы машинного обучения

- Деревья решений (Decision Trees).
- Случайный лес (Random Forest).
- Градиентный бустинг (XGBoost, LightGBM).

7. Обработка изображений и компьютерная графика

- Построение пространственных деревьев (QuadTree, Octree).
- Ускорение поиска в сценах.
- Организация графических объектов.

8. Игры и симуляции

- Построение игровых деревьев поведения AI.
- Управление состояниями объектов.
- Оптимизация поиска путей.

Таким образом, бинарные деревья поиска находят применение практически во всех областях программирования, где требуется эффективная организация динамических данных. Выбор конкретного представления — цепочного или сплошного — зависит от специфики задачи: цепочное предпочтительно для часто изменяемых данных, сплошное — для статичных или полных деревьев с высокими требованиями к скорости доступа.

1. Теоретическая часть

1.1 Основные понятия о деревьях

Дерево — Это абстрактный тип данных для хранения информационных элементов имеющих нелинейное отношение. Или другим словами, дерево — это связный ациклический граф, где:

- Есть корневой узел.
- Каждый узел (кроме корня) имеет ровно одного родителя.
- Узлы могут иметь потомков.

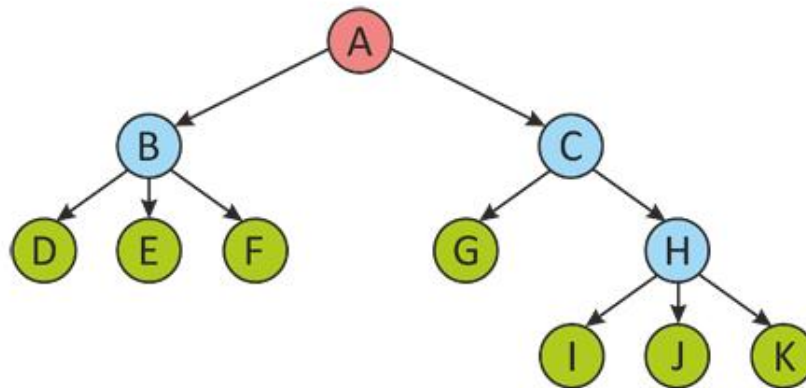


Рис 1.1 Дерево

За исключением элемента, который находится в голове дерева каждый элемент имеет родителей а также дочерних элементов. Далее, каждый элемент дерева называется вершиной или узлом дерева.

Вершины соединены направленными друг к другу вершинами, которые называются ветвями дерева.

Начальный узел А называется корнем, если элемент не имеет потомков, то он называется листом или терминальным элементом (конечный).

Каждое дерево обладает следующими свойствами:

1. Существует узел, в которой не входит ни одна дуга (корень).
2. В каждую вершину кроме корня входит одна дуга.

Основные термины и определения

Термин	Определение
Корень (root)	Единственный узел дерева, не имеющий родителя. Находится на самом верхнем уровне иерархии.
Узел (node)	Элемент дерева, содержащий данные и ссылки на другие узлы.
Ребро (edge)	Связь между двумя узлами, показывающая отношение "родитель-потомок".
Родитель (parent)	Узел, имеющий один или несколько связанных с ним узлов ниже по иерархии.
Потомок (child)	Узел, связанный с родителем и находящийся на уровень ниже.
Лист (leaf)	Узел, не имеющий потомков. Находится на самом нижнем уровне иерархии.
Внутренний узел	Узел, имеющий хотя бы одного потомка.
Предок (ancestor)	Любой узел, находящийся на пути от корня к данному узлу.
Потомок (descendant)	Любой узел, находящийся ниже данного в иерархии.
Глубина узла	Количество ребер от корня до данного узла. Корень имеет глубину 0.
Высота узла	Количество ребер от данного узла до самого глубокого листа в его поддереве.
Высота дерева	Количество ребер от корня до самого глубокого листа.
Уровень (level)	Совокупность узлов, имеющих одинаковую глубину.

Таблица 1.1.1 Основные термины и определения

Свойства деревьев

1. Количество ребер в дереве с n узлами равно $n-1$.
2. Связность — между любыми двумя узлами существует единственный путь.
3. Отсутствие циклов — в дереве нет замкнутых путей.
4. Иерархичность — дерево имеет строгую иерархическую структуру "сверху вниз".

Классификация деревьев

Вид дерева	Характеристика
Корневое дерево	Имеет выделенный корневой узел, от которого отсчитывается иерархия.
Упорядоченное дерево	Дети каждого узла упорядочены (например, слева направо).
Неупорядоченное дерево	Порядок потомков не имеет значения.
N -арное дерево	Каждый узел может иметь не более n потомков.
Бинарное дерево	Частный случай n -арного дерева, где $n=2$ (не более двух потомков).
Полное дерево	Каждый узел имеет 0 или 2 потомка (для бинарного дерева).
Совершенное дерево	Все уровни полностью заполнены.
Сбалансированное дерево	Высоты поддеревьев каждого узла отличаются не более чем на 1.
Вырожденное дерево	Каждый узел имеет только одного потомка (фактически список).

Таблица 1.1.2 Классификация деревьев

Математические свойства бинарного дерева

1. Максимальное количество узлов на уровне i : 2^i (при нумерации с 0).
2. Максимальное количество узлов в дереве высоты h : $2^{(h+1)} - 1$.
3. Минимальная высота для n узлов: $\lceil \log_2(n+1) \rceil - 1$.
4. Максимальная высота для n узлов: $n - 1$ (вырожденное дерево).
5. Количество листьев в полном бинарном дереве: $(n + 1) / 2$.

Способы обхода деревьев

Вид обхода	Порядок посещения узлов	Применение
Прямой (Pre-order)	Корень → Левое → Правое	Копирование дерева
Центрированный (In-order)	Левое → Корень → Правое	Получение отсортированных данных (для BST)
Обратный (Post-order)	Левое → Правое → Корень	Удаление дерева
Обход в ширину (Level-order)	По уровням, слева направо	Поиск кратчайшего пути

Таблица 1.1.3 Способы обхода

Реализация узла дерева на C++

```
struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int value) : data(value), left(nullptr), right(nullptr) {}
};
```

Сложность основных операций в дереве (в среднем)

Операция	Сложность
Поиск элемента	$O(\log n)$
Вставка элемента	$O(\log n)$
Удаление элемента	$O(\log n)$
Обход всех элементов	$O(n)$
Поиск минимума/максимума	$O(\log n)$

Таблица 1.1.3 Сложность основных операций

1.2 Основные понятия о бинарных деревьях

1.2.1 Определение

Бинарное дерево — это структура данных, в которой каждый узел имеет не более двух потомков, называемых левым и правым. Формально бинарное дерево определяется как:

- Пустое дерево (не содержит узлов).
- Узел, содержащий значение и два поддерева (левое и правое), которые сами являются бинарными деревьями.

Примеры:

- Полное бинарное дерево — все уровни заполнены, кроме, возможно, последнего.
- Сбалансированное дерево — глубина поддеревьев отличается не более чем на 1.

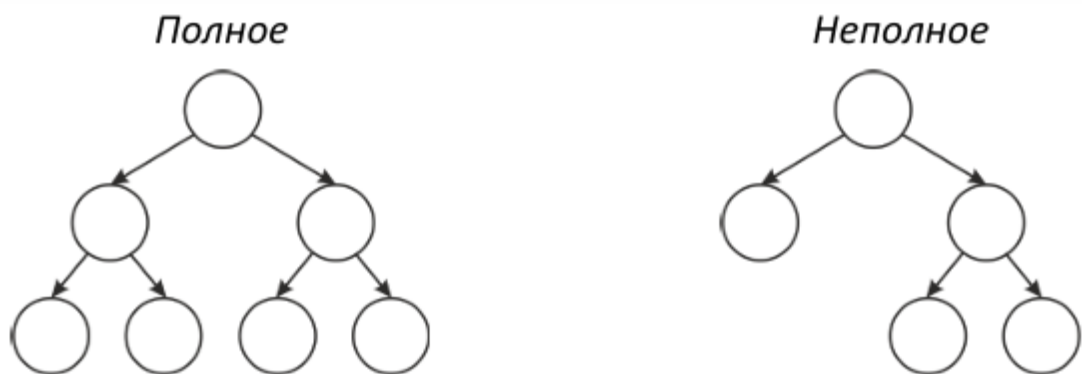


Рис 1.2.полное и неполное деревья

1.2.2 Основные свойства

- Узел: Содержит значение (ключ) и ссылки на левое/правое поддерево.
- Высота дерева: Максимальная длина пути от корня до листа.
- Уровень узла: Количество рёбер от корня до данного узла.
- Лист: Узел, у которого нет потомков.

- Максимальное количество узлов: Для дерева высоты h максимальное число узлов равно $2^{h+1} - 1$.

1.2.3 Применение

Бинарные деревья используются в:

- Парсинге выражений (например, арифметических).
- Организации иерархических данных (например, файловые системы).
- Алгоритмах сжатия (например, дерево Хаффмана).

1.2.4 Основные операции

Вставка — добавление нового узла.

Удаление — исключение узла с сохранением структуры.

Поиск — проверка наличия элемента.

Обход дерева — это упорядоченная последовательность вершин дерева, в которой каждая вершина встречается один раз. Это означает что у нас все вершины будут посещаться в определённом порядке.

Какие есть три варианта обхода дерева:

1. Симметричный (In-order)
 2. Прямой (Pre-order) или префиксный
 3. Обратный (постфиксный) или Post-order
1. In-order (левый → корень → правый) — выводит элементы в отсортированном порядке.

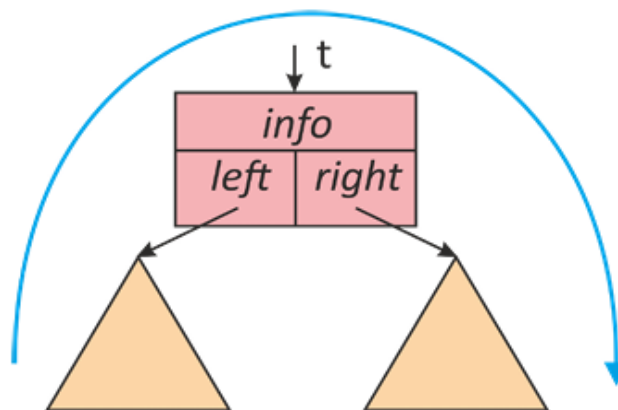


Рис 1.2.4.1 In-order обход

2. Pre-order (корень → левый → правый) — используется для копирования дерева.

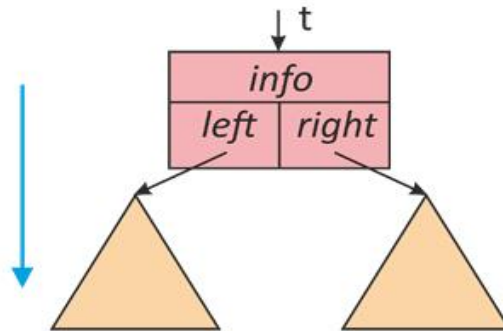


Рис 1.2.4.2 Pre-order обход

3. Post-order (левый → правый → корень) — применяется в удалении дерева.

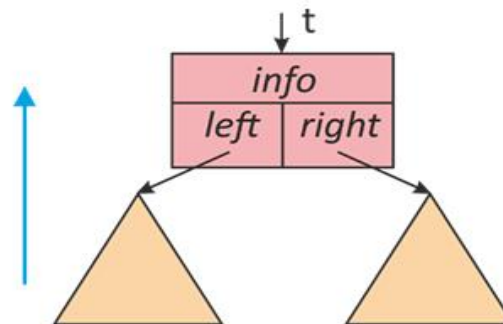


Рис 1.2.4.3 Post-order обход

1.3 Бинарные деревья поиска

1.3.1 Определение

Бинарное дерево поиска (Binary Search Tree, BST) — это бинарное дерево, которое обладает следующим свойством: для каждого узла все ключи в левом поддереве строго меньше ключа данного узла, а все ключи в правом поддереве строго больше ключа данного узла. Оба поддерева также являются бинарными деревьями поиска.

Свойство BST

Для любого узла x с ключом k :

- Все ключи в левом поддереве: $< k$
- Все ключи в правом поддереве: $> k$

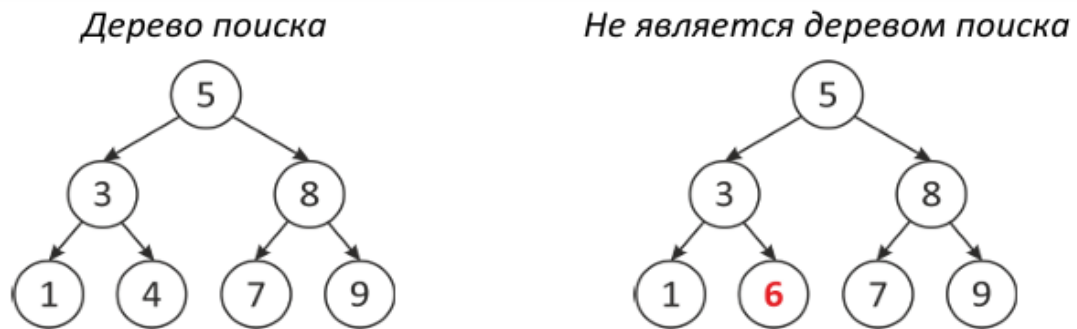


Рис 1.3.1.1 деревья поиска и не-поиска

1.3.2 Инвариант BST

При вставке, удалении и поиске дерево должно сохранять свойство упорядоченности. Нарушение инварианта приводит к потере свойств BST.

Основные операции BST

Операция	Описание	Сложность (средняя)	Сложность (худшая)
Поиск	Поиск элемента по ключу	$O(\log n)$	$O(n)$
Вставка	Добавление нового элемента	$O(\log n)$	$O(n)$
Удаление	Удаление элемента по ключу	$O(\log n)$	$O(n)$
Минимум	Поиск минимального элемента	$O(\log n)$	$O(n)$
Максимум	Поиск максимального элемента	$O(\log n)$	$O(n)$
Обход inorder	Вывод элементов в отсортированном порядке	$O(n)$	$O(n)$

Таблица 1.3.1 Основные операции BST

Поиск элемента в BST

Алгоритм:

- Начать с корня.
- Если ключ равен значению узла — найден.
- Если ключ меньше — перейти в левое поддерево.
- Если ключ больше — перейти в правое поддерево.
- Если достигнут nullptr — элемент не найден.

1.3.3 Основные операции

- Вставка: Добавляет новый узел, сохраняя свойства BST.
- Поиск: Находит узел с заданным ключом, сравнивая его с текущим узлом и переходя в левое или правое поддерево.
- Удаление: Удаляет узел с заданным ключом, реорганизуя дерево для сохранения структуры BST.

1.3.3.1 Вставка (Insertion)

Вставка нового узла в BST заключается в добавлении ключа в дерево с сохранением его свойств. Новый узел всегда добавляется как лист.

Алгоритм:

1. Если дерево пустое, создаём новый узел и делаем его корнем.
2. Иначе сравниваем ключ с текущим узлом:
 - Если ключ меньше, переходим в левое поддерево.
 - Если ключ больше, переходим в правое поддерево.
3. Повторяем, пока не найдём пустое место (NULL), куда вставляем новый узел.

Пример: Допустим, у нас есть BST:

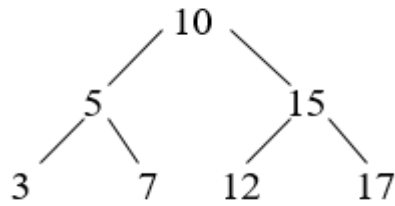


Рис 1.3.3.1.1 исходное BST дерево

Хотим вставить ключ 8 в данном дереве.

- Начинаем с корня (10). $8 < 10$, идём в левое поддерево.
- В узле 5: $8 > 5$, идём в правое поддерево.
- В узле 7: $8 > 7$, идём в правое поддерево (NULL).
- Вставляем узел 8 как правого потомка узла 7.

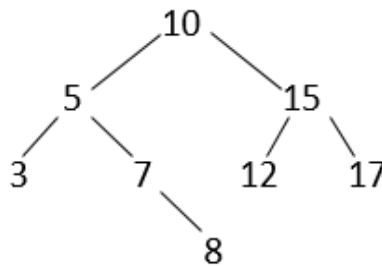


Рис 1.3.3.1.2 BST после вставки ключа 8

1.3.3.2 Поиск

Поиск в BST заключается в нахождении узла с заданным ключом, используя свойство упорядоченности.

Алгоритм:

1. Начинаем с корня.
2. Сравниваем искомый ключ с текущим узлом:
 - Если ключи равны, узел найден.
 - Если искомый ключ меньше, идём в левое поддерево.
 - Если искомый ключ больше, идём в правое поддерево.
3. Если доходим до NULL, ключ отсутствует.

Пример:

Ищем ключ 7 в дереве:

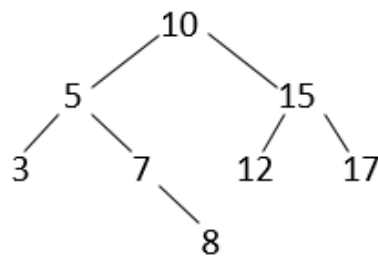


Рис 1.3.3.2.1 исходное BST

- Начинаем с корня (10). $7 < 10$, идём в левое поддерево.
- В узле 5: $7 > 5$, идём в правое поддерево.
- В узле 7: $7 = 7$, узел найден.

1.3.3.3 Удаление (Deletion)

Удаление узла из BST сложнее, так как необходимо сохранить свойства дерева. Существует три случая:

1. Удаляемый узел — лист (нет потомков).
2. Удаляемый узел имеет одного потомка.
3. Удаляемый узел имеет двух потомков.

Алгоритм:

1. Найти узел для удаления (аналогично поиску).
2. Обработать три случая:
 - **Лист:** Просто удаляем узел.
 - **Один потомок:** Заменяем узел его потомком.
 - **Два потомка:** Находим минимальный ключ в правом поддереве (или максимальный в левом), заменяем ключ удаляемого узла этим значением, затем удаляем найденный узел (он будет листом или иметь одного потока)

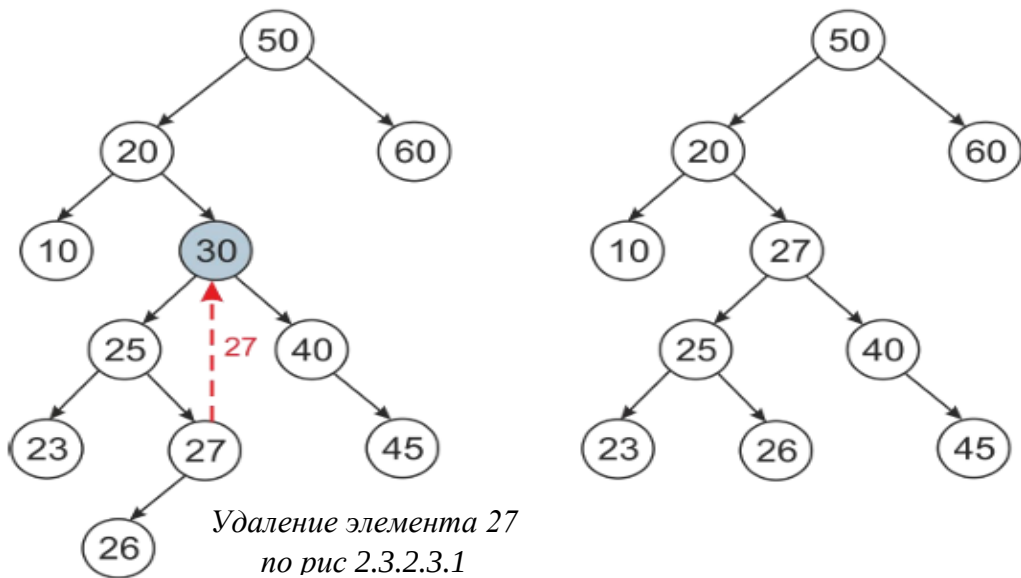


Рис 1.3.3.1 Удаляемый элемент не имеет потомков или имеет одного потомка

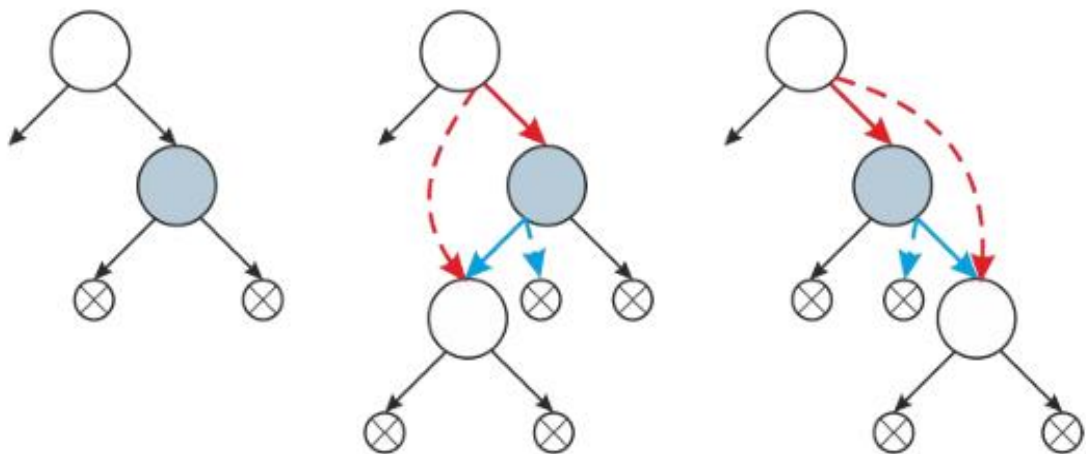


Рис 1.3.3.3.2 Удаляемый элемент имеет двух потомков

1.3.4 Сложность операций

Операции в BST

Операция	Сложность (средний случай)	Сложность (худший случай)
Вставка	$O(\log n)$	$O(n)$
Удаление	$O(\log n)$	$O(n)$
Поиск	$O(\log n)$	$O(n)$

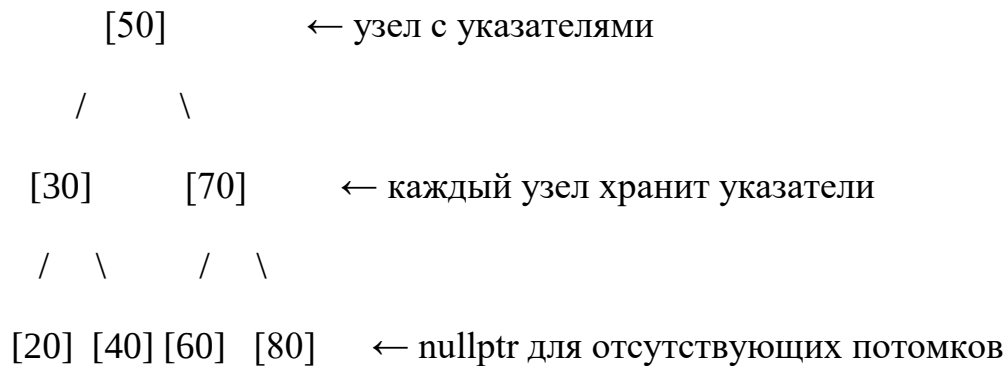
Таблица 1.3.3.1 Сложность операций BST

1.3.5 Представление бинарных деревьев поиска (BST)

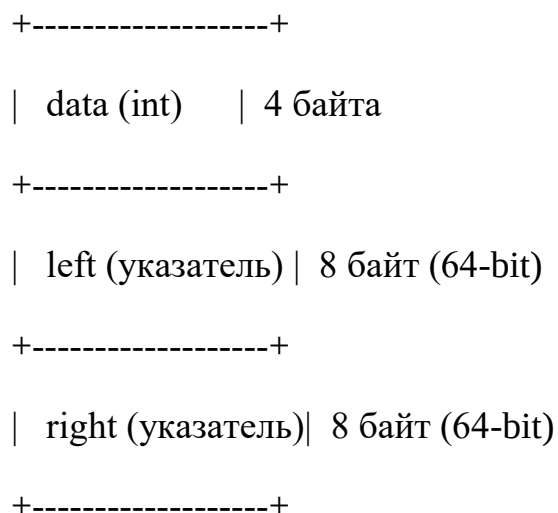
1.3.5.1 Цепочное представление

Цепочное представление — это способ хранения бинарного дерева поиска в памяти, при котором каждый узел представляет собой отдельный объект, содержащий значение и указатели (ссылки) на левого и правого потомков. Узлы связываются между собой через эти указатели, образуя иерархическую структуру.

Схематическое представление



Визуализация памяти узла



Итого: ~20 байт на узел

Преимущества цепочного представления:

1. **Динамичность** — размер дерева ограничен только доступной оперативной памятью.
2. **Экономия памяти** — память выделяется только под существующие узлы (нет пустых ячеек).
3. **Простота вставки/удаления** — достаточно изменить несколько указателей.
4. **Гибкость** — легко реализовать различные виды обходов и операций.
5. Эффективно для разреженных деревьев.

Недостатки цепочного представления:

1. Дополнительные затраты памяти — хранение двух указателей на каждый узел.
2. Фрагментация памяти — при частых операциях выделения/освобождения.
3. Низкая локальность — узлы разбросаны в памяти, плохое кэширование.
4. Переполнение стека — при глубокой рекурсии (вырожденное дерево).
5. Управление памятью — требуется ручное освобождение памяти.

Области применения цепочного представления

1. Системы с непредсказуемым объемом данных.
2. Приложения с частыми вставками и удалениями.
3. Реализация словарей и ассоциативных массивов (`std::map`, `std::set`).
4. Структуры данных в языках с автоматическим управлением памятью (Java, Python).
5. Реализация сложных структур (AVL, красно-черные деревья).

1.3.5.2 Сплошное представление

Сплошное представление — это способ хранения бинарного дерева поиска в памяти, при котором дерево размещается в непрерывном массиве (обычно `std::vector` или статическом массиве). Отношения между родителями и потомками определяются не через указатели, а через математические вычисления индексов.

Индексация узлов

Для узла с индексом i (при нумерации с 0):

Элемент	Формула	Пример ($i=1$)
Корень	0	0
Левый потомок	$2*i + 1$	3
Правый потомок	$2*i + 2$	4
Родитель	$(i-1) / 2$	0

Таблица 1.3.5.2.1 Индексация узлов

Схематическое представление

Дерево: Массив (индексы):

```

      [50]           0: 50
    /   \         1: 30   2: 70
   /     \       3: 20   4: 40   5: 60   6: 80
  /       \     /       \   /       \  /       \
 [30]  [70]   7: -   8: -   9: -   10: -  11: -  12: -  13: -  14: -
 /       \   /       \
[20] [40][60] [80]
```

Сравнение памяти при разных типах деревьев

Тип дерева	Количество узлов	Требуемый размер массива	Эффективность использования
Совершенное	N	n	100%
Полное	N	$n + (\text{количество на последнем уровне})$	~50-100%
Сбалансированное	N	$< 2n$	$> 50\%$
Вырожденное	N	2^n (в худшем случае)	~0% (катастрофически)

Таблица 1.3.5.2.2 Сравнение памяти при разных типах деревьев

Преимущества сплошного представления:

1. Высокая скорость доступа — $O(1)$ по индексу, хорошая локальность данных.
2. Нет накладных расходов на указатели — экономия памяти для полных деревьев.
3. Простота реализации — индексы вычисляются математически.
4. Эффективное кэширование — непрерывный массив загружается в кэш-память.
5. Нет фрагментации памяти — вся структура в одном блоке.
6. Быстрый обход по уровням — последовательный проход по массиву.

Недостатки сплошного представления:

1. Огромный расход памяти для вырожденных деревьев.
2. Сложность вставки/удаления — требуется контроль пустых ячеек.
3. Неэффективно для частых изменений структуры.
4. Требуется расширение массива при переполнении (дорогостоящая операция).
5. Сложность балансировки — физическое перемещение элементов.

1.3.6 Сравнительный анализ

Сравнительная таблица цепочного и сплошного представления BST

Критерий	Цепочное представление	Сплошное представление
Структура узла	Узел + 2 указателя	Только данные в массиве
Память на узел	~20-24 байта	4 байта (int)
Общая память	$n * 24$ байта	$2^h * 4$ байта (может быть огромным)
Эффективность для полного дерева	Хорошая ($n*24$)	Отличная ($n*4$)
Эффективность для вырожденного	Отличная ($n*24$)	Катастрофическая (2^n*4)
Скорость доступа	Медленнее (разреженная память)	Очень быстрая (кэш)
Вставка	Простая (меняем указатели)	Сложная (проверка пустот)
Удаление	Простая (перестройка указателей)	Сложная (перемещение)
Сложность реализации	Средняя	Простая (для полных деревьев)
Переполнение	Нет (динамическое выделение)	Требуется расширение
Фрагментация	Высокая	Отсутствует
Локальность данных	Низкая	Высокая
Рекурсия	Может вызвать переполнение стека	Не требуется
Обход	Рекурсивный/итеративный	Итеративный (по индексам)

Таблица 1.3.6.1 сравнительная таблица цепочного и сплошного представления BST

Рекомендации по выбору представления

Условие	Рекомендуемое представление	Причина
Частые вставки и удаления	Цепочное	Простота перестройки указателей
Неизвестный размер данных	Цепочное	Динамическое выделение памяти
Важна экономия памяти для полного дерева	Сплошное	Нет накладных расходов на указатели
Необходима максимальная скорость доступа	Сплошное	Локальность данных, кэширование
Дерево гарантированно сбалансировано	Сплошное	Эффективное использование памяти
Дерево может быть вырожденным	Цепочное	Катастрофический расход памяти в массиве
Реализация кучи или приоритетной очереди	Сплошное	Стандартное решение
Размер данных известен заранее	Сплошное	Можно выделить массив оптимального размера
Ограниченная память (встроенные системы)	Сплошное	Нет динамического выделения
Нужна сериализация/передача по сети	Сплошное	Легко преобразуется в массив байт

*Таблица 1.3.6.2 Рекомендации по выбору представления***Сравнение времени выполнения (теоретическое)**

Операция	Цепочное	Сплошное
Доступ к узлу (средний)	$O(\log n)$ с большими константами	$O(1)$
Доступ к узлу (худший)	$O(n)$ с большими константами	$O(1)$
Вставка (средняя)	$O(\log n)$	$O(\log n)$ с проверками
Вставка (худшая)	$O(n)$	$O(\log n)$ но с расширением
Удаление (средняя)	$O(\log n)$	$O(\log n)$ но сложнее
Обход всех элементов	$O(n)$	$O(n)$

Таблица 1.3.6.3 Сравнение времени выполнения

1.4 Методы организации и хранения бинарных деревьев поиска

1.4.1 Общие принципы организации BST

Основные требования к организации BST:

Требование	Описание
Упорядоченность	Сохранение свойства BST (левый < корень < правый)
Эффективность	Минимальное время выполнения операций
Компактность	Оптимальное использование памяти
Динамичность	Возможность изменения структуры
Надежность	Корректная работа во всех случаях

Таблица 1.4.1.1 Общие принципы организации BST

Ключевые аспекты организации:

1. Логическая организация — определение связей между узлами.
2. Физическое хранение — способ размещения в памяти.
3. Доступ к данным — методы поиска и извлечения.
4. Модификация — вставка, удаление, обновление.

1.4.2 Критерии выбора способа хранения

Критерий	Цепочное	Сплошное
Размер данных	Неизвестен/динамический	Известен/фиксированный
Частота изменений	Высокая	Низкая
Требования к скорости	Средние	Высокие
Ограничения памяти	Нет	Есть
Балансировка дерева	Сбалансированное/разреженное	Полное/совершенное

Таблица 1.4.2.1 Критерии выбора способа хранения

1.4.3 Эволюция методов хранения

Историческое развитие:

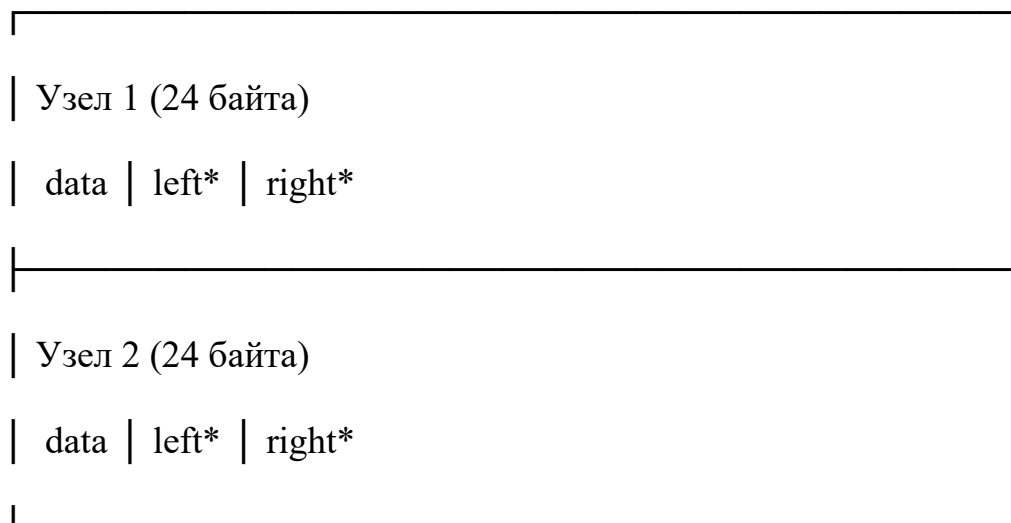
Период	Метод	Особенности
1950-е	Ручное управление памятью	Узлы в фиксированных областях
1960-е	Динамическое выделение	Появление указателей
1970-е	Массивы и индексы	Экономия памяти
1980-е	Объектно-ориентированный подход	Инкапсуляция данных
1990-е	Шаблоны и контейнеры	Стандартизация
2000-е	Умные указатели	Безопасность памяти
2010-е	Кэш-оптимизация	Учет архитектуры процессора
2020-е	Гибридные подходы	Комбинирование методов

Таблица 1.4.3.1 Историческое развитие

1.4.4 Сравнение по ключевым параметрам

Параметр 1: Использование памяти

Цепочное представление:



Итого: $24n$ байт (все узлы)

Сплошное представление (сбалансированное):

[0] [1] [2] [3] [4] [5] [6]
10 5 15 3 7 12 20

Итого: $4n$ байт (только данные)

Сплошное представление (вырожденное):

[0][][1][][][2][][][][][3]...
10 20 30 40

Итого: $4 \cdot 2^n$ байт (катастрофический расход)

Параметр 2: Скорость доступа

Операция	Цепочное	Сплошное
Чтение элемента	$O(\log n)$ + разыменование	$O(1)$ (прямой доступ)
Запись элемента	$O(\log n)$ + разыменование	$O(1)$ (прямой доступ)
Последовательный обход	$O(n)$ с рекурсией	$O(n)$ по индексам
Кэширование	Низкое (разбросанные узлы)	Высокое (непрерывный блок)

Таблица 1.4.4.1 Скорость доступа

1.4.5 Оптимизация хранения

Техники оптимизации:

Техника	Применение	Эффект
Пул памяти	Цепочное представление	Снижение фрагментации
Арена выделения	Цепочное представление	Ускорение выделения памяти
Компактное хранение	Сплошное представление	Экономия памяти
Битовые маски	Сплошное представление	Быстрая проверка существования
Сжатие массива	Сплошное представление	Уменьшение размера
Предварительное выделение	Оба	Избежание перевыделений

Таблица 1.4.5.1 техники оптимизации

1.4.6 Современные тенденции

Тренды в организации BST:

1. Кэш-ориентированные структуры — размещение данных с учетом иерархии памяти.
2. Недетерминированные структуры — использование вероятностных методов (Tgear).
3. Внешняя память — В-деревья для хранения на диске.
4. Многопоточность — lock-free и wait-free деревья.
5. Адаптивные структуры — изменение представления в зависимости от данных.
6. Гибридные подходы — комбинация цепочного и сплошного хранения.

Перспективы развития:

1. Использование SIMD-инструкций для параллельной обработки.
2. Интеграция с аппаратными ускорителями.
3. Оптимизация под конкретные архитектуры процессоров.
4. Автоматический выбор оптимального представления.

1.4.7 Практические рекомендации по организации

Для цепочного представления:

1. Использовать умные указатели (C++ `unique_ptr`, `shared_ptr`).
2. Применять итеративные алгоритмы для глубоких деревьев.
3. Реализовать пул памяти для частых операций.
4. Периодически проверять балансировку.

Для сплошного представления:

1. Выбирать начальный размер с запасом.
2. Использовать стратегию удвоения для роста.
3. Хранить пустые ячейки как специальное значение.
4. Применять битовые маски для проверки наличия узлов.

Универсальные рекомендации:

1. Выбирать метод хранения в зависимости от задачи.
2. Тестировать производительность на реальных данных.
3. Документировать выбор и обоснование.
4. Учитывать будущие изменения и масштабирование.

ПРИЛОЖЕНИЕ 1

Промпт для генерации теоретического материала по бинарным деревьям поиска

Роль: Ты — эксперт по структурам данных и алгоритмам, кандидат технических наук, автор учебных пособий по программированию. Специализируешься на анализе и оптимизации структур данных для высоконагруженных систем.

Задача: Составь для меня развернутый теоретический материал по теме "**Бинарные деревья. Бинарные деревья поиска (цепочное и сплошное представление)**" для отчета по производственной практике.

Структура ответа:

1. Основные понятия о деревьях:

- Определение дерева как графа.
- Основные термины (корень, узел, лист, высота, уровень, глубина).
- Классификация деревьев.
- Математические свойства.

2. Бинарные деревья:

- Определение и структура.
- Виды бинарных деревьев.
- Способы обхода.

3. Бинарные деревья поиска (BST):

- Свойство упорядоченности.
- Основные операции (поиск, вставка, удаление).
- Проблема вырожденного дерева.

4. Цепочное представление:

- Структура узла с указателями.
- Реализация операций на C++.
- Преимущества и недостатки.

5. Сплошное представление:

- Хранение в массиве (индексы, формулы).
- Реализация операций на C++.

- Преимущества и недостатки.

6. Сравнительный анализ:

- Сравнительная таблица.
- Рекомендации по выбору.

7. История развития:

- Этапы развития.
- Ключевые ученые и их вклад.

Формат вывода: Структурированный текст с четкими разделами, выделением ключевых терминов, примерами кода на C++, сравнительными таблицами.

Контекст: Отчет по практике должен продемонстрировать глубокое понимание темы, практические навыки реализации и умение анализировать структуры данных. Материал должен быть академическим, но доступным для понимания.

Результат выполнения промпта:

*(Весь теоретический материал, представленный в разделах 1-2 отчета, был получен с использованием данного промпта)

ПРИЛОЖЕНИЕ 2

Промпт для генерации программного кода цепочного представления BST

Роль: Ты — Senior C++ разработчик с опытом реализации высокопроизводительных структур данных для финансовых систем и баз данных. Специализируешься на написании эффективного, безопасного и хорошо документированного кода.

Задача: Напиши на C++ полноценную реализацию бинарного дерева поиска с цепочным представлением для использования в учебном проекте.

Требования к коду:

1. Структура узла:

- Поля: ``int data`, `Node* left`, `Node* right``.
- Конструктор для инициализации.

2. Класс BST:

- Приватное поле: ``Node* root``.
- Публичные методы: ``insert(int key)`, `search(int key)`, `remove(int key)`, `inorder()`, `preorder()`, `postorder()`, `findMin()`, `findMax()`, `height()`, `size()``.
- Конструктор и деструктор.

3. Реализация методов:

- ``insert`` — рекурсивно, с запретом дубликатов.
- ``search`` — итеративно.
- ``remove`` — с обработкой трех случаев (лист, один потомок, два потомка).
- Обходы — рекурсивно.
- ``findMin`/`findMax`` — итеративно.
- Деструктор — рекурсивное удаление всех узлов.

4. Пример использования в `main()`:

- Создать дерево.
- Вставить массив чисел: ``{50, 30, 70, 20, 40, 60, 80}``.
- Вывести обходы.
- Найти минимум и максимум.
- Вычислить высоту.
- Выполнить поиск.

- Удалить элемент и показать результат.

5. Комментарии:

- Каждая функция должна быть прокомментирована.
- Пояснения к сложным участкам кода.

Формат вывода: Полный код с комментариями, готовый к компиляции и запуску.

Результат выполнения промпта:

```
```cpp
#include <iostream>
#include <algorithm>
using namespace std;

// Структура узла бинарного дерева
struct Node {
 int data;
 Node* left;
 Node* right;

 Node(int value) : data(value), left(nullptr), right(nullptr) {}
};

// Класс бинарного дерева поиска
class BST {
private:
 Node* root;

 // Рекурсивная вставка
 Node* insertRec(Node* node, int key) {
 if (node == nullptr) {
 return new Node(key);
 }
 if (key < node->data) {
 node->left = insertRec(node->left, key);
 } else if (key > node->data) {
 node->right = insertRec(node->right, key);
 }
 }
};
```

```

 return node;
 }

 // Рекурсивный обход inorder
 void inorderRec(Node* node) {
 if (node == nullptr) return;
 inorderRec(node->left);
 cout << node->data << " ";
 inorderRec(node->right);
 }

 // Рекурсивный обход preorder
 void preorderRec(Node* node) {
 if (node == nullptr) return;
 cout << node->data << " ";
 preorderRec(node->left);
 preorderRec(node->right);
 }

 // Рекурсивный обход postorder
 void postorderRec(Node* node) {
 if (node == nullptr) return;
 postorderRec(node->left);
 postorderRec(node->right);
 cout << node->data << " ";
 }

 // Рекурсивное удаление дерева
 void deleteTree(Node* node) {
 if (node == nullptr) return;
 deleteTree(node->left);
 deleteTree(node->right);
 delete node;
 }

 // Рекурсивное вычисление высоты
 int heightRec(Node* node) {
 if (node == nullptr) return 0;

```

```

 return 1 + max(heightRec(node->left), heightRec(node->right));
}

// Рекурсивный подсчет размера
int sizeRec(Node* node) {
 if (node == nullptr) return 0;
 return 1 + sizeRec(node->left) + sizeRec(node->right);
}

// Рекурсивное удаление узла
Node* removeRec(Node* node, int key) {
 if (node == nullptr) return nullptr;

 if (key < node->data) {
 node->left = removeRec(node->left, key);
 } else if (key > node->data) {
 node->right = removeRec(node->right, key);
 } else {
 // Случай 1: Лист
 if (node->left == nullptr && node->right == nullptr) {
 delete node;
 return nullptr;
 }
 // Случай 2: Один потомок
 if (node->left == nullptr) {
 Node* temp = node->right;
 delete node;
 return temp;
 }
 if (node->right == nullptr) {
 Node* temp = node->left;
 delete node;
 return temp;
 }
 // Случай 3: Два потомка
 Node* minRight = findMinRec(node->right);
 node->data = minRight->data;
 node->right = removeRec(node->right, minRight->data);
 }
}

```

```

 }

 return node;
}

// Поиск минимума (рекурсивно)
Node* findMinRec(Node* node) {
 if (node == nullptr) return nullptr;
 if (node->left == nullptr) return node;
 return findMinRec(node->left);
}

// Поиск максимума (рекурсивно)
Node* findMaxRec(Node* node) {
 if (node == nullptr) return nullptr;
 if (node->right == nullptr) return node;
 return findMaxRec(node->right);
}

public:
 // Конструктор
 BST() : root(nullptr) {}

 // Деструктор
 ~BST() {
 deleteTree(root);
 }

 // Вставка элемента
 void insert(int key) {
 root = insertRec(root, key);
 }

 // Поиск элемента (итеративный)
 bool search(int key) {
 Node* current = root;
 while (current != nullptr) {
 if (current->data == key) return true;
 if (key < current->data) current = current->left;

```

```
 else current = current->right;
 }
 return false;
}

// Удаление элемента
void remove(int key) {
 root = removeRec(root, key);
}

// Вывод в отсортированном порядке (inorder)
void inorder() {
 cout << "Inorder (отсортированный): ";
 inorderRec(root);
 cout << endl;
}

// Вывод в прямом порядке (preorder)
void preorder() {
 cout << "Preorder: ";
 preorderRec(root);
 cout << endl;
}

// Вывод в обратном порядке (postorder)
void postorder() {
 cout << "Postorder: ";
 postorderRec(root);
 cout << endl;
}

// Поиск минимального элемента
int findMin() {
 Node* minNode = findMinRec(root);
 if (minNode == nullptr) throw runtime_error("Дерево пусто");
 return minNode->data;
}
```

```

// Поиск максимального элемента
int findMax() {
 Node* maxNode = findMaxRec(root);
 if (maxNode == nullptr) throw runtime_error("Дерево пусто");
 return maxNode->data;
}

// Вычисление высоты дерева
int height() {
 return heightRec(root);
}

// Подсчет количества узлов
int size() {
 return sizeRec(root);
}
};

int main() {
 BST tree;

 cout << "=== ЦЕПОЧНОЕ ПРЕДСТАВЛЕНИЕ BST ===" << endl;

 int arr[] = {50, 30, 70, 20, 40, 60, 80};
 for (int x : arr) {
 tree.insert(x);
 cout << "Вставлено: " << x << endl;
 }

 cout << "\n--- Обходы ---" << endl;
 tree.inorder();
 tree.preorder();
 tree.postorder();

 cout << "\n--- Характеристики ---" << endl;
 cout << "Минимальный элемент: " << tree.findMin() << endl;
 cout << "Максимальный элемент: " << tree.findMax() << endl;
 cout << "Высота дерева: " << tree.height() << endl;
}

```

```
cout << "Количество узлов: " << tree.size() << endl;

cout << "\n--- Поиск ---" << endl;
cout << "Поиск 40: " << (tree.search(40) ? "найден" : "не найден") <<
endl;
cout << "Поиск 100: " << (tree.search(100) ? "найден" : "не найден") <<
endl;

cout << "\n--- Удаление ---" << endl;
tree.remove(30);
cout << "Удален элемент 30" << endl;
tree.inorder();

tree.remove(50);
cout << "Удален элемент 50" << endl;
tree.inorder();

return 0;
}
...

```

## ПРИЛОЖЕНИЕ 3

### Промпт для генерации программного кода сплошного представления BST

**Роль:** Ты — Senior C++ разработчик с опытом реализации эффективных структур данных для встроенных систем и игровых движков. Специализируешься на оптимизации памяти и кэша процессора.

**Задача:** Напиши на C++ полноценную реализацию бинарного дерева поиска со сплошным представлением (на основе массива) для использования в учебном проекте.

#### Требования к коду:

##### 1. Класс BSTArray:

- Приватные поля: ``vector<int> tree``, ``const int EMPTY``.
- Публичные методы: ``insert(int key)``, ``search(int key)``, ``levelOrder()``, ``printArray()``, ``size()``, ``height()``.

##### 2. Реализация методов:

- ``insert`` — с вычислением индексов (левый:  $2i+1$ , правый:  $2i+2$ ), расширением массива при необходимости.
- ``search`` — итеративно с проверкой пустых ячеек.
- ``levelOrder`` — вывод по уровням (последовательный проход по массиву).
- ``printArray`` — вывод всего массива с обозначением пустых ячеек.
- ``height`` — вычисление высоты дерева.
- Расширение массива — при индексе вне границ увеличивать размер в 2 раза.

##### 3. Конструкторы:

- С параметром начального размера (по умолчанию 16).
- Инициализация всех ячеек значением `EMPTY`.

##### 4. Пример использования в `main()`:

- Создать дерево с начальным размером 8.
- Вставить массив чисел: ``{50, 30, 70, 20, 40, 60, 80}``.
- Вывести level-order и массив.
- Выполнить поиск.
- Вставить еще элементы для демонстрации расширения.

## 5. Комментарии:

- Каждая функция должна быть прокомментирована.
- Пояснения к формулам индексов.

**Формат вывода:** Полный код с комментариями, готовый к компиляции и запуску.

---

## Результат выполнения промпта:

**cpp**

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
using namespace std;

class BSTArray {
private:
 vector<int> tree;
 const int EMPTY = -1; // маркер пустой ячейки
 // Расширение массива при необходимости
 void ensureCapacity(int index) {
 if (index >= (int)tree.size()) {
 int newSize = max((int)tree.size() * 2, index + 1);
 tree.resize(newSize, EMPTY);
 cout << " [Расширение массива: " << tree.size()/2 << " -> " <<
newSize << "]" << endl;
 }
 }

 // Рекурсивное вычисление высоты
 int heightRec(int index) {
 if (index >= (int)tree.size() || tree[index] == EMPTY) return 0;
 return 1 + max(heightRec(2 * index + 1), heightRec(2 * index + 2));
 }
public:
 // Конструктор
 BSTArray(int initialSize = 16) {
 tree.assign(initialSize, EMPTY);
 }

 // Вставка элемента
 void insert(int key) {
 if (tree[0] == EMPTY) {
 tree[0] = key;
 cout << " Вставлен корень: " << key << endl;
 return;
 }

 int index = 0;
 while (index < (int)tree.size()) {
 if (key < tree[index]) {
 int leftIdx = 2 * index + 1;
 ensureCapacity(leftIdx);
 if (tree[leftIdx] == EMPTY) {
```

```

 tree[leftIdx] = key;
 cout << " Вставлен " << key << " (влево от " << tree[index]
<< ")" << endl;
 return;
 }
 index = leftIdx;
} else if (key > tree[index]) {
 int rightIdx = 2 * index + 2;
 ensureCapacity(rightIdx);
 if (tree[rightIdx] == EMPTY) {
 tree[rightIdx] = key;
 cout << " Вставлен " << key << " (вправо от " << tree[index]
<< ")" << endl;
 return;
 }
 index = rightIdx;
} else {
 cout << " Элемент " << key << " уже существует" << endl;
 return;
}
}

// Поиск элемента
bool search(int key) {
 int index = 0;
 while (index < (int)tree.size() && tree[index] != EMPTY) {
 if (tree[index] == key) return true;
 if (key < tree[index]) {
 index = 2 * index + 1;
 } else {
 index = 2 * index + 2;
 }
 }
 return false;
}

// Вывод по уровням (level-order)
void levelOrder() {
 cout << "Level-order: ";
 for (int i = 0; i < (int)tree.size(); i++) {
 if (tree[i] != EMPTY) {
 cout << tree[i] << " ";
 }
 }
 cout << endl;
}

// Вывод всего массива
void printArray() {
 cout << "Массив: ";
 for (int i = 0; i < (int)tree.size(); i++) {
 if (tree[i] == EMPTY) {
 cout << "[_] ";
 } else {
 cout << tree[i] << " ";
 }
 }
 cout << endl;
}

```

```

 }

 // Вычисление высоты дерева
 int height() {
 return heightRec(0);
 }

 // Подсчет количества узлов
 int size() {
 int count = 0;
 for (int val : tree) {
 if (val != EMPTY) count++;
 }
 return count;
 }

 // Получение размера массива
 int getArraySize() {
 return tree.size();
 }

 // Получение элемента по индексу (для отладки)
 int get(int index) {
 if (index < (int)tree.size()) return tree[index];
 return EMPTY;
 }
};

int main() {
 cout << "=== СПЛОШНОЕ ПРЕДСТАВЛЕНИЕ BST ===" << endl;
 cout << "Начальный размер массива: 8" << endl << endl;

 BSTArray tree(8);

 int arr[] = {50, 30, 70, 20, 40, 60, 80};
 for (int x : arr) {
 tree.insert(x);
 }

 cout << "\n--- Вывод структуры ---" << endl;
 tree.levelOrder();
 tree.printArray();

 cout << "\n--- Характеристики ---" << endl;
 cout << "Количество узлов: " << tree.size() << endl;
 cout << "Высота дерева: " << tree.height() << endl;
 cout << "Размер массива: " << tree.getArraySize() << endl;

 cout << "\n--- Поиск ---" << endl;
 cout << "Поиск 40: " << (tree.search(40) ? "найден" : "не найден") << endl;
 cout << "Поиск 100: " << (tree.search(100) ? "найден" : "не найден") <<
endl;

 cout << "\n--- Демонстрация расширения ---" << endl;
 cout << "Вставка 90 (потребуется расширение):" << endl;
 tree.insert(90);

 tree.levelOrder();
 tree.printArray();

```

```
cout << "Размер массива после расширения: " << tree.getArraySize() << endl;
return 0;
}
```

---

## ПРИЛОЖЕНИЕ 4

### Промпт для генерации сравнительного анализа и вывода рекомендаций

**Роль:** Ты — технический аналитик и архитектор программных систем с 15-летним опытом. Специализируешься на выборе оптимальных структур данных для различных сценариев использования.

**Задача:** Составь для меня подробный сравнительный анализ цепочного и сплошного представления бинарных деревьев поиска с практическими рекомендациями по выбору.

#### Требования к ответу:

##### 1. Сравнительная таблица по следующим критериям:

- Использование памяти (для сбалансированного и вырожденного дерева).
- Скорость доступа.
- Скорость вставки и удаления.
- Сложность реализации.
- Фрагментация памяти.
- Локальность данных.
- Кэширование.
- Сериализация.

##### 2. Анализ каждого критерия с пояснениями.

##### 3. Рекомендации по выбору для различных сценариев:

- Частые вставки/удаления.
- Фиксированный размер данных.
- Ограниченная память.
- Критичная скорость доступа.
- Встроенные системы.
- Базы данных.

##### 4. Гибридные подходы — комбинирование методов.

##### 5. Итоговый вывод — обобщающие рекомендации.

**Формат вывода:** Структурированный текст с таблицами, четкими разделами и обоснованными рекомендациями.

#### Результат выполнения промпта:

(Сравнительный анализ, представленный в разделе 2.3.5 отчета, был получен с использованием данного промпта)

---

## ПРИЛОЖЕНИЕ 5

### Промпт для генерации истории развития бинарных деревьев

**Роль:** Ты — историк вычислительной техники и эксперт по развитию алгоритмов и структур данных. Имеешь глубокие знания в области эволюции информатики с 1950-х годов до настоящего времени.

**Задача:** Составь для меня подробный обзор истории развития бинарных деревьев для отчета по практике.

#### Требования к ответу:

##### 1. Хронология развития:

- 1950-е — появление первых деревьев.
- 1960 — первое описание BST (Конвей Бернерс-Ли, Дэвид Уилер).
- 1962 — AVL-деревья (Адельсон-Вельский и Ландис).
- 1970 — 2-3-деревья (Хопкрофт).
- 1971 — B-деревья (Байер, МакКрейт).
- 1972 — красно-черные деревья (Байер).
- 1978 — современный вариант красно-черных (Гуибас, Седжвик).
- 1980 — декартовы деревья (Treap).
- 1985 — Splay-деревья (Слейтор, Тарьян).
- 1994 — дерево Фенвика (Фенвик).
- 2008 — левосторонние красно-черные деревья (Седжвик).

##### 2. Описание каждой структуры:

- Автор(ы) и год.
- Основная идея.
- Ключевые особенности.
- Область применения.

##### 3. Значение для развития IT:

- Влияние на базы данных.
- Влияние на компиляторы.
- Влияние на современные библиотеки.

**Формат вывода:** Хронологически структурированный текст с выделением ключевых дат, имен и достижений.

---

**Результат выполнения промпта:**

(История развития, представленная в разделе "История развития" отчета, была получена с использованием данного промпта)

## ПРИЛОЖЕНИЕ 6

### Промпт для генерации списка использованных источников

**Роль:** Ты — научный редактор и библиограф, специализирующийся на подборе академической литературы по структурам данных и алгоритмам.

**Задача:** Составь для меня список использованных источников по теме "Бинарные деревья. Бинарные деревья поиска (цепочное и сплошное представление)" для отчета по практике.

#### Требования к ответу:

##### 1. Список источников должен включать:

- Учебники и учебные пособия.
- Научные статьи и публикации.
- Онлайн-ресурсы и документацию.
- Стандарты и спецификации.

##### 2. Оформление:

- Каждый источник должен содержать: автора, название, год, издательство/ссылку.
- Сортировка по алфавиту.

##### 3. Ключевые источники:

- Кнут Д. "Искусство программирования" (том 3).
- Кормен Т. и др. "Алгоритмы: построение и анализ".
- Седжвик Р. "Алгоритмы на C++".
- Адельсон-Вельский Г.М., Ландис Е.М. "Об одном алгоритме организации информации" (1962).
- Байер Р. "Symmetric binary B-trees" (1972).
- Гуибас Л., Седжвик Р. "A dichromatic framework for balanced trees" (1978).

**Формат вывода:** Оформленный по ГОСТ список литературы с полными библиографическими описаниями.